Databases and its Applications

Advanced NoSQL and Emerging Database trends

Pooja T S

Computer Applications

Databases and its Applications

Advanced NoSQL and Emerging Database trends

Aggregation Framework in MongoDB: $match, $group, $project

Pooja T S

Computer Applications

Databases and its Applications

Aggregation Operators: Overview

- MongoDB provides a rich set of **aggregation operators** used inside aggregation pipeline stages.
- These operators define:
  - How values are compared
  - How data is calculated
  - How expressions are evaluated
- Operators are executed within pipeline stages such as `$match`, `$group`, and `$project`.

Databases and its Applications

Categories of Aggregation Operators

- Aggregation operators are broadly classified into:
  - Query Comparison Operators
  - Logical Operators
  - Accumulator Operators
  - Arithmetic and Expression Operators
- Each category serves a specific purpose in data processing.

Databases and its Applications

Comparison Operators

- Comparison operators are used to compare field values.
- Common comparison operators:
  - `$eq` — equal to
  - `$ne` — not equal to
  - `$gt`, `$gte`
  - `$lt`, `$lte`
- Primarily used inside the **$match** stage.

Databases and its Applications

Logical Operators

- Logical operators combine multiple conditions.
- Frequently used logical operators:
  - `$and`
  - `$or`
  - `$not`
  - `$nor`
- Logical operators are commonly nested within **$match**.

Databases and its Applications

Accumulator Operators

- Accumulator operators perform calculations over grouped documents.
- Used exclusively within the **$group** stage.
- Common accumulator operators:
  - `$sum`
  - `$avg`
  - `$min`
  - `$max`
  - `$push`

Databases and its Applications

Expression and Arithmetic Operators

- Expression operators compute derived values.
- Common expression operators:
  - `$add`, `$subtract`
  - `$multiply`, `$divide`
  - `$concat`
- These operators are frequently used in **$project**.

Databases and its Applications

Operators vs Pipeline Stages

- Pipeline stages define **what** operation is performed.
- Operators define **how** the operation is executed.
  - **$match** uses comparison and logical operators
  - **$group** uses accumulator operators
  - **$project** uses expression operators
- Operators cannot exist independently of pipeline stages.

Databases and its Applications

From Operators to Aggregation Framework

- Aggregation operators form the building blocks of the pipeline.
- Understanding operators is essential before studying:
  - **$match**
  - **$group**
  - **$project**
- We now move to the Aggregation Framework in detail.

Databases and its Applications

Aggregation Framework in MongoDB

- The Aggregation Framework is used to **process and transform data** stored in MongoDB collections.
- It works using a **pipeline model** where documents pass through multiple stages.
- Each stage performs a specific operation on the documents.

Databases and its Applications

Aggregation Pipeline

- An aggregation pipeline is a **sequence of stages**.
- Each stage:

  takes input documents,

  processes them,

  passes output to the next stage.
- Common stages used:

  `$match`

  `$group`

  `$project`

Databases and its Applications

Aggregation Syntax

- General syntax of aggregation:

```
db.students.aggregate([ stage1, stage2, stage3 ])
```

- Each stage is written as a document.
- Stages are executed in the order specified.

Databases and its Applications

$match Stage

- **$match** is used to **filter documents**.
- It works similar to the `find()` method.
- Only documents satisfying the condition move to the next stage.

Databases and its Applications

$match Example

- Task: Retrieve students belonging to the MCA department.

```
db.students.aggregate([ {$match:{
department:"MCA" }} ])
```

- Output includes only MCA students.

Databases and its Applications

$group Stage

- **$group** is used to **group documents** based on a field.
- It is commonly used for:
  - counting documents,
  - calculating averages,
  - computing totals.
- Grouping key is specified using `_id`.

Databases and its Applications

$group Example: Count

- Task: Count number of students in each department.

```
db.students.aggregate([ {$group:{
_id:"$department", totalStudents:{$sum:1} }} ])
```

- Output shows department-wise student count.

Databases and its Applications

$group Example: Average

- Task: Find average CGPA per department.

```
db.students.aggregate([ {$group:{
_id:"$department", avgCGPA:{$avg:"$cgpa"} }}])
```

- Computes academic performance by department.

Databases and its Applications

$project Stage

- **$project** is used to **select and reshape fields**.
- It can:

  include fields,

  exclude fields,

  rename fields,

  create computed fields.

Databases and its Applications

$project Example

- Task: Display student name and CGPA only.

```
db.students.aggregate([ {$project:{ _id:0,
name:1, cgpa:1 }} ])
```

- Removes unnecessary fields from output.

Databases and its Applications

Combining $match and $group

- Aggregation stages can be combined.
- Task: Count MCA students only.

```
db.students.aggregate([ {$match:{
department:"MCA" }}, {$group:{ _id:"$department",
count:{$sum:1} }}])
```

Databases and its Applications

Combining $group and $project

- Task: Show department name and average CGPA with clear labels.

```
db.students.aggregate([ {$group:{
_id:"$department", avg:{$avg:"$cgpa"} }},
{$project:{ _id:0,
department:"$id" , avgCGPA : "$avg" }}])
```

Databases and its Applications

Why Aggregation?–Instead of find()

- `find()` retrieves documents.
- Aggregation:

  performs calculations,

  summarizes data,

  produces analytical reports.
- Essential for reporting and analytics.

Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 391